UNDERSTANDING SELECTION METHODS

CCE 109/L

CCE 102/L

Objectives

- Identify the selection methods used in Java Programming
- Describe how each selection methods used.
- Create a program applying the selection methods.

What is a selection/conditional statement?

- Selection - selection is used to make choices depending on information.
- An algorithm can be made smarter by using IF, THEN, and ELSE functions to reiterate instructions, or to move the process in question to different parts of the program.

Java has the following conditional statements:

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

Logical condition statement supported by Java:

- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b
- Equal to: a == b
- Not Equal to: a != b

IF STATEMENTS

The If statements

The if statements is used to test the condition. It only yields two results, true or false. There are four types of if statements in java:

- If statements
- If-Else Statements
- Nested If
- If-Else-If Ladder

If Statement Single Selection

- Use the if statement to specify a block of Java code to be executed if a condition is true.

Example:

- if(bank balance is zero) borrow money
- if(room is dark) put on lights
- if(today is Sunday) lets play football
- if(student mark is 60) you passed
- if(code is 0) person is male
- if(age is more than 55) person is retired

If Statement Single Selection

- For example, suppose the passing grade of each student is 75. The pseudocode statement is

```
if(students grade is greater than or equal to 75)
        print passed
```

This determines whether the condition is greater than or equal to 75 is true or false.

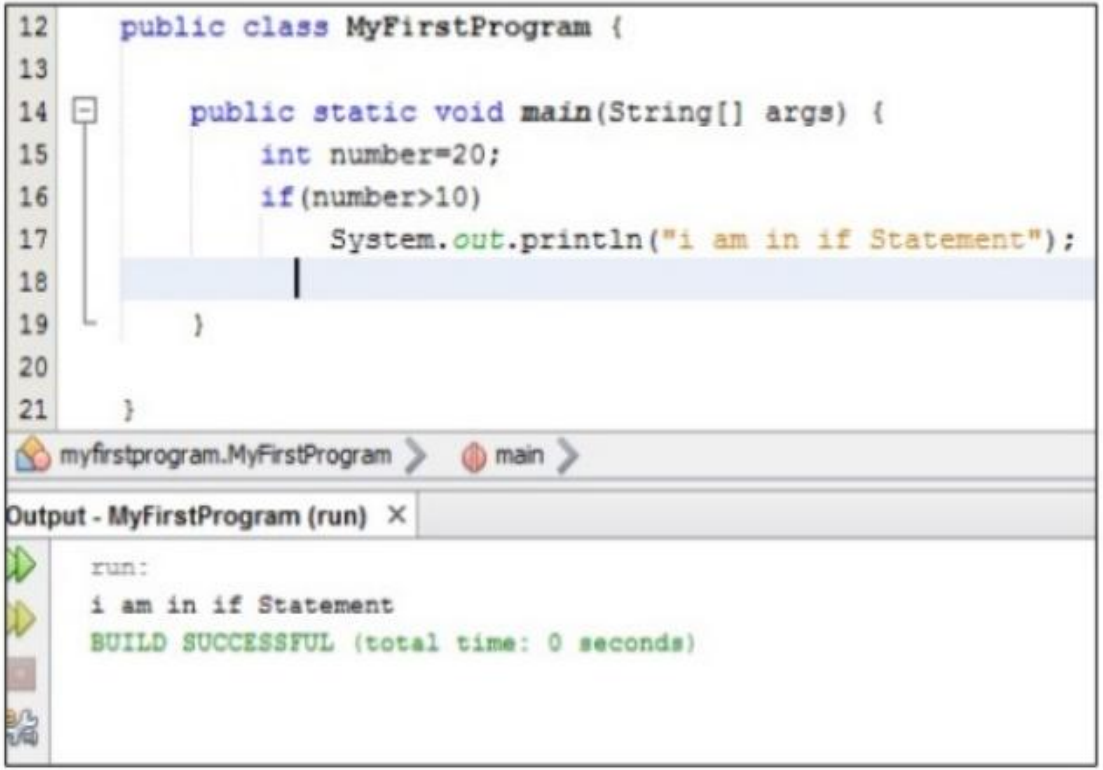
Explanation:

- If the condition is true, passed in printed and next statement after the code is performed.
- If the condition is false, the print statement is ignored and next statement after the code is performed.

Syntax:

```
if (condition) {
// block of code to be executed if the condition
is true
}
```

Example:


12 public class MyFirstProgram {
13
14 public static void main(String[] args) {
15 int number=20;
16 if(number>10)
17 System.out.println("i am in if Statement");
18
19 }
20
21 }
myfirstprogram.MyFirstProgram
main
Output - MyFirstProgram (run)
run:
i am in if Statement
BUILD SUCCESSFUL (total time: 0 seconds)

Example:

```
12  public class MyFirstProgram {
13
14      public static void main(String[] args) {
15          int a=10,b=20;
16              if (a<b) {
17
18                  System.out.println("This is if statement example");
19
20              }
21
22              }
23
24  }
```

myfirstprogram.MyFirstProgram

Output - MyFirstProgram (run)

```
run:
This is if statement example
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example: Will the output be displayed?

```
int a=20, b=10;
        if (a<=b){
            System.out.println("a is greater
than b.");
        }
```

Explanation: No. Variable a is not less than b. Hence, the condition is false. The statement within the curly braces will not be executed.

Example: Will the output be displayed?

```
int a=20, b=10;
        if (a>=b){
            b+=12;//variable b will be added 12.
          System.out.println("The value of b is: " +b);
        }
```

Expected Output: “The value of b is 22.”

The else Statement

- Use the else statement to specify a block of code to be executed if the condition is false.

Syntax

```
if (condition) {
// block of code to be executed if the condition is true
} else {
// block of code to be executed if the condition is false }
```

Sample Code:

```
int a=75;
     if (a>=75){
         System.out.println("You passed.");}
      else{
        System.out.println("You failed.");
 }
```

Output: You passed.

The else if Statement

- Use the `else if` statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {
 // block of code to be executed if condition1 is true
} else if (condition2) {
// block of code to be executed if the condition1 is false
and condition2 is true
} else {
// block of code to be executed if the condition1 is false
and condition2 is false }
```

Example:

```
int grade=74;
  if (grade>=96){
     System.out.println("Your Grade is an A");
 } else if (grade>=90){
      System.out.println("Your Grade is an B");
 } else if(grade>=80){
      System.out.println("Your Grade is an C");
 } else if (grade>=75){
      System.out.println("Your Grade is an D");
 } else{
      System.out.println("You failed.");
 }
```

Expected output: You failed.

Nested if statements

- It is always legal to nest if-else statements which means you can use one if or else statements inside another if-else statement.

Syntax:

```
if(boolean expression 1)
     if(Boolean expression2)
          //statements
```

Example:

```
int x=30,y=10;
 if (x==30){
      if(y==10){
           System.out.println("This statement is
executed.");
      }
 }
```

Expected Output: This statement is executed.

Accepting input from the user

- One of the strengths in java is the huge libraries available to you.
- This is the code written to help you do specific jobs. One class that is very useful is the Scanner class.
- It allows to accepts inputs from the user.
- To use the class, you must first import the class itself using the code, import `java.util.Scanner`.
- This import statement must be above the class declaration.

Accepting input from the user

To create a new Scanner object this must be the code written:

```
Scanner input = new Scanner(System.in);
```

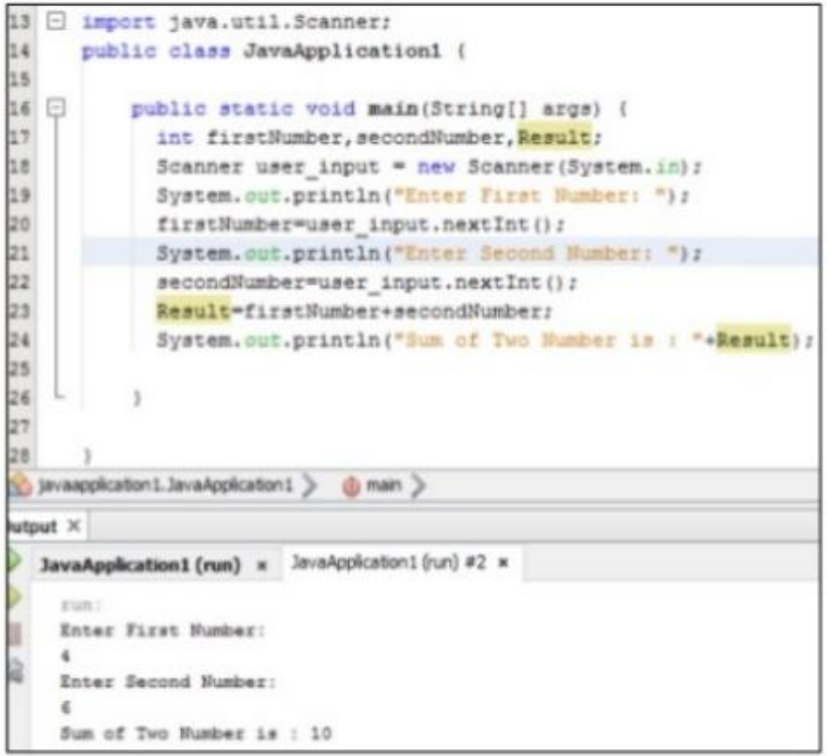
To get the user input, you call one of the action of the many methods available to your scanner object. One of these methods is called next.

```
Int firstNumber;
firstNumber = input.next();
```

Example: Accepting input from user

```
int firstnum;
      Scanner user_input=new Scanner (System.in);
      System.out.println("Enter First number: ");
      firstnum=user_input.nextInt();
       System.out.println("The value of firstnum
is: "+ firstnum);
```

Example: Accepting input from user



```
13  import java.util.Scanner;
14  public class JavaApplication1 {
15
16      public static void main(String[] args) {
17        int firstNumber,secondNumber,Result;
18        Scanner user_input = new Scanner(System.in);
19        System.out.println("Enter First Number: ");
20        firstNumber=user_input.nextInt();
21        System.out.println("Enter Second Number: ");
22        secondNumber=user_input.nextInt();
23        Result=firstNumber+secondNumber;
24        System.out.println("Sum of Two Number is : "+Result);
25
26      }
27
28  }
```

javaapplication1.JavaApplication1 > main >

utput ×

JavaApplication1 (run) × | JavaApplication1 (run) #2 ×

```
run:
Enter First Number:
4
Enter Second Number:
6
Sum of Two Number is : 10
```

Other methods to read other types of input:

Method	Description
`nextBoolean()`	Reads a `boolean` value from the user
`nextByte()`	Reads a `byte` value from the user
`nextDouble()`	Reads a `double` value from the user
`nextFloat()`	Reads a `float` value from the user
`nextInt()`	Reads a `int` value from the user
`nextLine()`	Reads a `String` value from the user
`nextLong()`	Reads a `long` value from the user
`nextShort()`	Reads a `short` value from the user